

서울 랜드마크 이미지 분류 프로젝트 보고서

[C팀] 박수빈 김연수 김소연

1. 데이터 이해

1.1. train.csv와 test.csv의 역할

(1) train.csv

학습에 사용하는 파일. 이미지 파일명(file_name)과 정답 라벨(label, 0~9)이 함께 포함되어 있어 모델이 "이 이미지는 어떤 클래스인지" 학습하는 데 사용된다.

(2) test.csv

학습이 완료된 모델이 예측해야 할 이미지 목록. 파일명(file_name)만 있고 정답 라벨은 없다. 모델이 예측한 결과를 submission 파일로 만들 때 사용한다.

1.2. train 이미지와 test 이미지의 차이

구분	train 이미지	test 이미지
용도	모델 학습 및 검증	최종 예측 대상
정답 라벨	있음 (0~9)	없음
이미지 수	723장	199장
활용	학습 578장	submission_cnn.csv, submission_transfer.csv 생성에 사용

1.3. 클래스 개수 및 데이터 개수

항목	내용
클래스 수	10개 (라벨 0~9, 각 숫자가 서울 랜드마크 하나에 해당)
전체 학습 데이터	723개
학습용	578개 (80%)
검증용	145개 (20%)
테스트 데이터	199개

1.4. 이미지 크기 및 색상 채널 처리 방식

(1) 이미지 크기 처리 방식

데이터 셋 내 이미지들은 크기가 제각각이다. 모델은 입력 이미지 크기가 통일되어야 하므로, 전처리 단계에서 transforms.Resize((224, 224))를 적용하여 모든 이미지를 224×224 픽셀로 통일한다.

(2) 색상 채널 처리 방식

색상 채널은 Image.open(img_path).convert('RGB')를 사용하여 흑백, RGBA 등 다양한 형식의 이미지를 모두 RGB 3채널로 통일한다. 이를 통해 모든 이미지가 (3, 224, 224) 형태로 모델에 입력된다.

항목	처리 방식	결과
이미지 크기	<code>transforms.Resize((224, 224))</code>	224 × 224 픽셀로 통일
색상 채널	<code>.convert('RGB')</code>	R, G, B 3채널로 통일
최종 입력 shape	<code>ToTensor()</code> 적용 후	(3, 224, 224)

2. 전체 실행 흐름 설명

2.1. 데이터 불러오기

- pandas의 `read_csv()`를 사용하여 `train.csv`, `test.csv`, `sample_submission.csv`를 불러온다. CSV 파일을 표(DataFrame) 형태로 변환해야 파이썬 코드에서 파일명, 라벨 등의 정보를 다룰 수 있다.
- 단계에서 클래스 수(10개), 학습 데이터(723개), 테스트 데이터(199개)를 확인한다.

2.2. 이미지 경로 생성

- CSV에는 001.PNG처럼 파일명만 있어 실제 이미지를 열 수 없다. `os.path.join()`을 사용하여 파일명 앞에 폴더 경로를 붙여 전체 경로(file_path)를 생성한다.
(예: 001.PNG → /content/drive/MyDrive/dataset/train/001.PNG 이 경로가 있어야 CustomDataset에서 이미지를 실제로 열 수 있다.)

2.3. Dataset 구성

- train 데이터를 80(학습):20(검증)으로 분리한 후 CustomDataset 클래스를 구현한다.
- PyTorch 모델은 이미지를 직접 받을 수 없어, 이미지를 꺼내는 방법을 정의한 Dataset이 필요하다. `__len__()`은 전체 개수를 반환하고, `__getitem__()`은 idx번째 이미지를 열어 전처리 후 반환한다. `is_test` 파라미터로 train/val과 test를 구분하여 하나의 클래스로 모두 처리한다.

2.4. DataLoader 구성

- 이미지를 한 장씩 모델에 넘기면 너무 오래 걸리기 때문에 DataLoader로 32장씩 묶어 배치 단위로 전달한다.
- 학습용은 `shuffle=True`로 매 epoch마다 순서를 섞어 모델이 순서를 외우는 것을 방지하고, 검증/테스트용은 `shuffle=False`로 성능을 일관되게 측정한다. `num_workers=2`로 이미지 로딩 속도를 높인다.

2.5. 모델 정의

- `nn.Module`을 상속받아 독자적인 CNN 구조(CNNModel)를 정의합니다. 이미지의 공간적 특징을 뽑는 부분과 이를 분류하는 분류기 부분으로 역할을 나눕니다.

2.6. Loss/Optimizer 설정

- 다중 클래스 분류에 적합한 `nn.CrossEntropyLoss`를 손실함수로 정의하고, 가중치를 효율적으로 최적화하기 위해 Adam 옵티마이저(학습률:0.001)를 채택합니다.

2.7. 학습

10 에포크 동안 순전파로 예측값을 구하고, 손실 계산 후 역전파를 통해 가중치 그래디언트를 구해 모델을 갱신합니다.

2.8. 검증

매 에포크 종료 시 가중치를 고정된 상태(`torch.no_grad()`)에서 검증 데이터 셋의 정확도를 측정하여 모델의 성능 추이와 과적합 여부를 감시합니다.

2.9. 테스트 예측

학습이 끝난 모델을 평가 모드(`model.eval()`)로 전환한 뒤, 테스트 데이터 셋 통과 후 가장 확률이 높은 클래스 인덱스를 추출하여 저장합니다.

2.10. 제출 파일 생성

예측된 테스트 결과를 샘플 제출 파일의 라벨 컬럼에 입력하고 `submission_cnn.csv` 파일로 내보내어 최종 결과물을 완성합니다.

3. 기본 CNN 모델 설명

3.1. CNN 구조

크게 특징 추출부(self.features)와 분류부(self.classifier)의 2단계 구조로 설계되었습니다. Convolution 블록 3개를 통과하며 이미지 특징을 압축하고, 직렬화 후 Fully Connected Layer를 거쳐 클래스를 예측합니다.

3.2. Conv2d의 역할

이미지 위를 커널(필터)이 슬라이딩하며 공간적 특징(에지, 텍스처 등)을 추출합니다. 코드에서는 커널 크기 3, 패딩 1을 주어 출력의 가로·세로 크기를 유지하면서 채널 수만 확장($3 > 32 > 64 > 128$)시켰습니다.

3.3. ReLU의 역할

$f(x) = \max(0, x)$ 성질을 가진 비선형 활성화 함수입니다. 선형 결합 구조인 신경망에 비선형성을 부여하여 모델이 복잡한 패턴을 학습할 수 있게 만들고, 역전파 시 그래디언트 소실 문제를 방지합니다.

3.4. MaxPool2d의 역할

지정된 영역 내에서 가장 큰 값만 남기는 다운샘플링 기법입니다. 특성 맵의 해상도를 절반으로 줄여 연산량을 대폭 감소시키고, 이미지 내 개체의 미세한 위치 변화에도 안정적으로 인식할 수 있게 합니다..

3.5. Flatten이 필요한 이유

3차원 형태의 특성 맵 Tensor를 분류기인 Dense 레이어에 입력하기 위해서는 1차원의 긴 벡터 형태로 한 줄로 펼쳐주어야 하기 때문에 필수적입니다.

3.6. 마지막 Linear layer 출력 크기를 10으로 설정한 이유

문제에서 다루는 최종 타깃 클래스의 개수가 총 10개이기 때문입니다. 모델의 최종 출력은 각 클래스에 매핑되는 10개의 Logit 값이 되어야 CrossEntropyLoss를 통해 확률 분포로 변환할 수 있습니다.

4. 전이학습 모델 설명

4.1. ResNet18을 선택한 이유

ResNet18은 이미지 분류 분야에서 가장 널리 사용되는 기본 모델로, VGG처럼 단순히 레이어를 쌓는 구조와 달리 잔차 연결을 통해 깊은 네트워크에서도 기울기 소실 문제 없이 안정적으로 학습할 수 있습니다. EfficientNet이나 MobileNet 등 더 경량화된 모델도 있지만, ResNet이 더 직관적이고 코드 작성 시 참고할 수 있는 자료들이 많아 모델로 선택했습니다.

4.2. pretrained 사용 및 classifier layer 수정

pretrained=True 로 ImageNet 사전학습 가중치를 불러온 뒤, 원래 1,000개 클래스를 분류하도록 설계된 마지막 fully connected layer를 이번 프로젝트의 클래스 수인 10개에 맞게 교체했습니다.

4.3. freeze 전략 및 학습 방식

두 가지 전략을 비교 실험했습니다.

먼저, Feature Extraction은 backbone 전체를 freeze하고 마지막 classifier layer만 학습하는 방식으로, 사전학습된 특징 추출 능력을 그대로 활용합니다.

Fine-tuning은 freeze 없이 전체 레이어를 학습하는 방식으로, 사용하는 데이터 셋에 더 세밀하게 적응할 수 있습니다. 단, Fine-tuning은 학습률이 너무 높으면 사전 학습된 가중치가 손상될 수 있어 lr=0.0001로 낮춰서 실험했습니다.

4.4. 전이 학습의 장단점

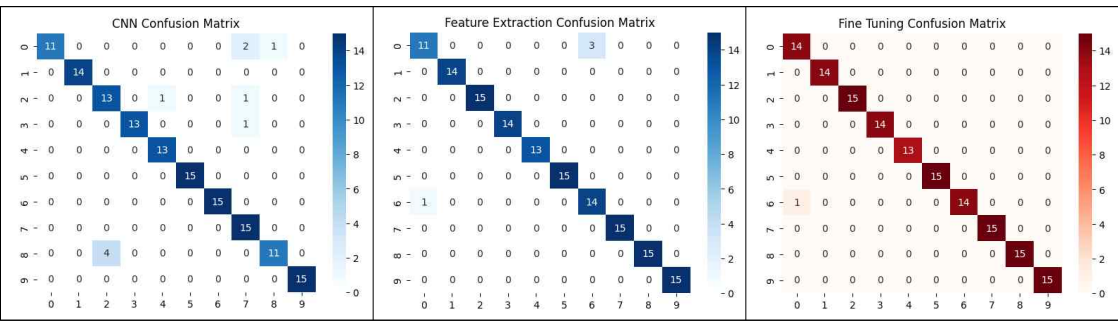
장점으로는 적은 데이터로도 높은 성능을 낼 수 있고 학습 시간이 단축됩니다. 반면 사전 학습 데이터와 우리 데이터의 특성이 많이 다를 경우 성능이 제한될 수 있고, 모델 구조를 자유롭게 변경하기 어렵다는 단점이 있습니다.

5. 실험 결과 비교

5.1. 과제 1, 2 결과 비교

실험 번호	모델	pretrai ned	freeze 전략	Optim izer	LR	Batch Size	Augment ation	Best Val Acc
Exp1	CNN	x	x	Adam	0.001	32	Horizont alFlip, Rotation (10)	0.9310
Exp2	ResNet18	o	전체		0.0001	32		0.9724
Exp3	ResNet18	o	없음		0.0001	32		0.9931

5.2. 성능 개선 실험1 - Confusion Matrix 및 오분류 분석



모델이 전체적으로 좋은 성능을 보이긴 했지만, CNN에서는 총 9개의 오분류가 발생하였고, Feature Extraction에선 총 4개의 오분류가 발생하였습니다. Fine Tuning은 1개만이 오분류 되어 가장 좋은 성능을 보였습니다.

특히 CNN에서 클래스8에서 4개가 클래스 2로 오분류 되었는데, 이는 클래스8과 클래스2가 유사한 외관을 가지고 있어서 이러한 결과가 발생하였다고 추측할 수 있습니다.



[오분류 원인 확인을 위한 클래스별 이미지 하나씩 출력한 결과]

반면, 전이 학습 모델은 사전학습된 특징 추출 능력을 바탕으로 훨씬 잘 구분해낸 것을 확인할 수 있습니다.

5.3. pretrained 사용 여부 비교

```
*** Feature Extraction Val Acc : 33.10%  
    Fine-tuning Val Acc      : 47.59%
```

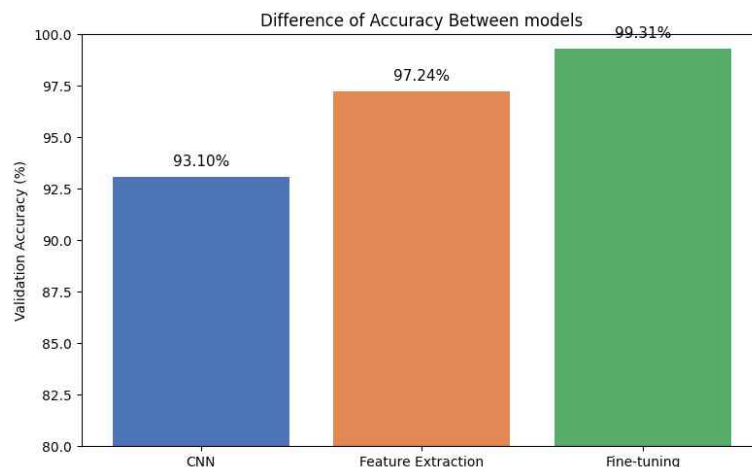
[pretrained=False 결과]

동일한 ResNet18 구조에서 pretrained 사용 여부에 따라 성능 차이를 비교하였습니다. pretrained=True일 때 Feature Extraction 97.24%, Fine-tuning 99.31%를 기록한 반면, pretrained=False일 때는 각각 33.10%, 47.59%로 크게 낮은 성능을 보였습니다.

이는 학습 데이터가 578장으로 매우 적은 환경에서 처음부터 학습하는 것만으로는 충분한 특징 추출 능력을 갖추기 어렵기 때문입니다. 반면 ImageNet의 대규모 데이터로 사전 학습된 가중치를 활용하면 이미 풍부한 시각적 특징 추출 능력을 갖춘 상태에서 시작하므로, 적은 데이터로도 높은 성능을 낼 수 있습니다. 이번 실험을 통해 데이터 수가 적은 환경일수록 pretrained 가중치의 활용이 결정적인 역할을 한다는 것을 확인할 수 있었습니다.

6. 결과 해석

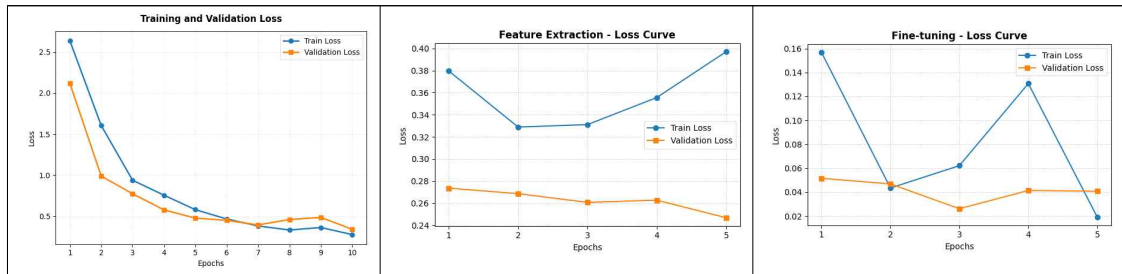
1) 모델 성능 비교 및 성능 차이 발생 이유



결과적으로 Fine-tuning 방식의 ResNet18이 가장 높은 성능(0.9931)을 보였고, Feature Extraction(0.9724), CNN(0.9310)이 뒤를 이었습니다.

ResNet18을 사용한 모델이 비교적 성능이 좋은 이유는 CNN의 경우 처음부터 직접 설계한 모델로 주어진 데이터 셋만으로 학습했기 때문에 특징 추출 능력이 비교적 제한됩니다. 하지만 ResNet18같은 경우 이미 대규모 데이터 셋으로 사전학습 된 특징 추출 능력을 활용할 수 있어 더 좋은 성능을 보여주었습니다.

2) train loss와 validation loss를 비교했을 때 과적합이 발생했는가?



CNN의 경우 Train Loss와 Val Loss가 초반에 함께 감소하다가 후반부로 갈수록 Train Loss는 계속 낮아지는 반면 Val Loss는 상대적으로 높게 유지되는 양상을 보였습니다. 이는 과적합 징후가 나타나는 것으로 보입니다.

Feature Extraction의 경우 Val Loss가 전 구간에 걸쳐 꾸준히 감소하는 안정적인 학습 곡선을 보였습니다. Train Loss가 5epoch에서 소폭 상승하였으나 Val Loss는 계속 낮아지고 있어 과적합 없이 잘 학습되었음을 확인할 수 있습니다.

Fine-tuning의 경우 Val Loss가 전체적으로 매우 낮은 수준(0.02~0.05)을 유지하였고, Train Loss도 빠르게 수렴하였습니다. 4epoch에서 Train Loss가 일시적으로 상승하는 구간이 있었으나 Val Loss는 안정적으로 유지되어 학습 과정이 전반적으로 안정적이었음을 확인할 수 있습니다. 세 모델 중 가장 낮은 Loss와 가장 안정적인 학습 곡선을 보였습니다.

3) augmentation은 성능에 어떤 영향을 주었는가?

학습 데이터에 RandomHorizontalFlip과 RandomRotation(10도)을 적용하였습니다. 데이터 수가 578장으로 적은 상황에서 증강을 통해 모델이 다양한 방향과 각도의 이미지를 학습할 수 있게 하여 과적합을 억제하는 데 도움을 주었습니다. 검증/테스트 데이터에는 증강을 적용하지 않아 실제 예측 환경과의 일관성을 유지했습니다.

4) pretrained 사용 여부는 어떤 차이를 만들었는가?

pretrained=False로 설정하여 실험한 결과 Val Acc가 30~40% 수준에 머물렀습니다. 반면 pretrained=True를 사용한 경우 97% 이상의 성능을 보였습니다. 학습 데이터가 578장으로 매우 적은 환경에서는 사전학습된 가중치의 특징 추출 능력이 결정적인 역할을 한다는 것을 확인할 수 있었습니다.

5) 성능 개선을 위해 추가로 시도할 수 있는 방법은 무엇인가?

첫째, Early Stopping을 적용하여 Val Loss가 최저점인 시점에서 학습을 조기 종료하면 과적합을 방지할 수 있습니다. 둘째, ColorJitter, RandomErasing 등 더 다양한 데이터 증강 기법을 적용하여 모델의 일반화 성능을 높일 수 있습니다. 셋째, Learning Rate Scheduler를 활용하면 학습 후반부에 학습률을 점진적으로 낮춰 더 안정적인 수렴을 기대할 수 있습니다.

7. 팀원별 기여도

이름	담당 역할	실제 수행 내용	이해한 핵심 개념
박수빈	팀원 C	전이학습 모델 구현, 성능 비교, 오분류 분석 및 제출 파일 생성 Confusion Matrix 및 오분류 분석	전이학습의 필요성, 혼동행렬, CNN 모델 구조 등
김소연	팀원 B	기본 CNN 모델 구현, 학습 과정 분석, 하이퍼파라미터 튜닝 pretrained 사용 여부 비교	CNN 모델 구성 및 pretrained사용여부로 인한 결과 차이
김연수	팀원 A	데이터 구조 분석, CSV 처리, Dataset/DataLoader 구현, 데이터 증강 실험	데이터 전처리 개념 및 실제 처리 방식에 대한 전반적인 이해

8. 생성형 AI 사용 기록

박수빈	1. 코랩에서 깃을 사용하는 방법에 대한 부분(연동, pull/push하는 방법 등) -> 코랩에서는 !을 앞에 쓰고 터미널에서 사용하는 코드를 작성하면 된다. -> 이외에도 연결과정에 발생한 다양한 오류 및 충돌 등의 문제 해결 시 활용하였습니다. 2. 실습 전 CIFAR-10모델로 연습(실제로 해본지가 오래되어서 실습 전 연습이 필요하다고 느낌.) -> 작성해준 커리큘럼을 바탕으로 연습 진행. CNN모델 작성하여 사용하는 것과 전이 학습의 차이를 이해하게 됨. -> 더불어 이때 사용한 코드를 기반으로 주어진 팀프로젝트 데이터셋을 활용할 수 있도록 수정하여 사용함. 3. 과제 요구사항이 잘 반영되었는지 코드를 전체적으로 확인 -> 실제로 부족한 부분에 대한 피드백을 받았고(CNN과의 결과 비교) 받은 결과를 바탕으로 추가적으로 코드를 작성함. 4. 그래프 등 결과 시각화코드 작성 -> 작성해준 코드를 바탕으로 원하는 방식으로 커스텀하여(색상 변경) 사용
김소연	<코랩과 깃 파일 연결> 1. 질문-> 코랩에서 깃에 연결한 파일을 여는 방법을 알려줘 2. 연결 방법과 연결 시 필요한 명령어를 알게 되었습니다. <적합한 parameter> 1. 질문사항: 해당 데이터셋에 적합한 optimizer와 batch사이즈를 알려줘. 2. 받은 도움: 코드 짤 때 알려준 걸로 사용했습니다
김연수	

전체 과제 설명을 통해 과제에서 요구하는 것과 필요 역량 확인하기

- Q. 서울 랜드마크 이미지 데이터셋을 활용하여 이미지 분류 모델을 구현하고자 해. 기본 CNN 모델과 전이학습 기반의 모델 성능을 비교할거야.
- A. 크게 두 가지 방향으로 진행하면 돼.
기본 CNN 모델: 직접 Conv2d, ReLU, MaxPool2d 등을 쌓아서 모델을 구현하는 방식이야. 처음부터 학습하기 때문에 데이터가 적으면 성능이 낮을 수 있어.
전이학습 기반 모델: ResNet, EfficientNet 등 ImageNet으로 미리 학습된 모델을 가져와서 마지막 분류 레이어만 우리 데이터(10개 클래스)에 맞게 바꾸는 방식이야. 적은 데이터로도 높은 성능을 낼 수 있어.

항목	기본CNN	전이학습
학습 방식	처음부터 학습	사전 학습 모델 활용
데이터 요구량	많이 필요	적어도 가능
학습 속도	느림	빠름
기대 성능	낮음	높음

과제1 중 1번

- Q. `train.csv`, `test.csv`, `sample_submission.csv` 이 무엇이고, 이것들을 불러오려면 어떻게 해야해?
- A. `train.csv`는 학습에 사용하는 파일이야. 이미지 파일명(file_name)과 정답 라벨(label, 0~9)이 함께 있어서 모델이 "이 이미지는 어떤 클래스인지" 학습하는 데 사용돼.
`test.csv`는 모델이 예측해야 할 이미지 목록이야. 파일명만 있고 정답 라벨은 없어. 학습이 완료된 모델로 예측한 결과를 submission 파일로 만들 때 사용해.
`sample_submission.csv`는 제출 파일의 형식 예시야. 실제 정답이 아니라 "이런 형식으로 제출해"를 보여주는 템플릿이야.
이것들을 불러오기 위해서는 pandas 라이브러리의 `read_csv()`를 사용해서 불러올 수 있어.
또한 `pd.read_csv()`는 CSV 파일을 읽어서 파이썬에서 다룰 수 있는 표(DataFrame) 형태로 변환해줘.
- Q. 이 모델을 구현하려면 드라이브에 저장된 학습용/검증용 이미지를 불러와야하는데, 이걸 어떻게 구현할 수 있어?
- A. `BASE_DIR`은 파일이 저장된 구글 드라이브 폴더 경로야.